QUANTUM COMPUTING · REINFORCEMENT LEARNING · HPC

$|\psi\rangle$

Sintesi di circuiti quantistici con PPO, Transformer e HPC

Alessandro La Mantia

Dalla ricerca casuale all'apprendimento di circuiti brevi

IL PERCORSO DEL PROGETTO

Indice

Dall’obiettivo del progetto ai risultati: baseline, primi training, correzioni e possibili miglioramenti.

01 Obiettivo e metodo
Slide 3–9 · Target, ambiente, reward, Adam, PPO e Transformer

02 Ciclo di addestramento e HPC
Slide 10–11 · Step, episodi, iterazioni, run e cluster

03 Prima del training: baseline casuale
Slide 12–16 · Bell, GHZ, Random 2 e 3 qubit; risultati iniziali

04 Primi training e problema GHZ
Slide 17–21 · Bell, blocco a zero e soluzioni

05 Training aggiornato e grafici
Slide 22–27 · GHZ, Random 2 e 3 qubit; confronti e reward

06 Come migliorare l’agente
Slide 28 · Possibili sviluppi del training

01 · OBIETTIVO

Imparare a costruire un circuito

Partire da |00…0⟩ e preparare uno stato puro target con alta fidelity e il minor numero possibile di gate.

Perché questo approccio è così importante e da “High Performance Computing”?

1 · Complessità combinatoria

Il problema è combinatorio ed enorme

Aumentando i qubit crescono le possibili scelte di gate e dei qubit su cui applicarli. Con la profondità del circuito, il numero di sequenze cresce esponenzialmente.

Esplorare tutte le combinazioni diventa rapidamente impraticabile: l’agente cerca strutture utili senza enumerarle tutte.

2 · Hardware quantistico NISQ

I computer quantistici reali sono rumorosi

Nell’era NISQ, le operazioni sui qubit sono imperfette. Più gate e circuiti più lunghi possono accumulare errori e degradare la qualità del risultato.

Cercare il **minor numero di gate e la massima fedeltà** rende la sintesi rilevante anche per l’esecuzione su hardware reale.

3 · Risorse HPC

Servono molte risorse di calcolo

L’agente deve effettuare molti tentativi: ciascuno richiede di simulare il circuito e ottimizzare gli angoli, oltre al costo di addestrare il Transformer.

I cluster HPC, come quello dell’Unipr, offrono risorse per training prolungati e worker paralleli che raccolgono esperienze per la policy.

Il rumore NISQ motiva la ricerca di circuiti brevi; gli esperimenti di questo progetto usano simulazioni classiche di stati puri, senza un modello di rumore.

Quale stato vogliamo raggiungere?

2 qubit

Bell

$|\Phi^+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$

3 qubit negli esperimenti

GHZ

$|GHZ_3\rangle = (|000\rangle + |111\rangle) / \sqrt{2}$

2 e 3 qubit

Target random

Vettore complesso normalizzato con 2^n ampiezze.

Richiede di adattare ampiezze e fasi, oltre all'entanglement.

$$F = |\langle \psi_{target} \mid \psi_{circuito} \rangle|^2 \qquad d = 1 - F$$

La fidelity varia tra 0 e 1. Negli esperimenti si considera successo **F ≥ 0,99**: è un'approssimazione accurata, non necessariamente uno stato esatto.

Target caricabili da JSON o NumPy; validazione e normalizzazione in quantum_rl/1.targets.py. L'ambiente termina con d < 0,01; i report classificano il successo con F ≥ 0,99.

02 · AMBIENTE GYMNASIUM

Una mossa aggiunge un gate

Gymnasium definisce il ciclo RL; PennyLane simula lo stato prodotto dal circuito.

Gate utilizzati: RX, RY, RZ, CNOT

- **RX, RY, RZ:** ruotano un qubit attorno agli assi X, Y e Z. Gli angoli sono parametri continui ottimizzati internamente.
- **CNOT:** inverte il qubit target quando il controllo è nello stato |1⟩; può creare entanglement.

Nella baseline, H permette di creare la sovrapposizione necessaria per Bell e GHZ: senza H, usando solo X, Y, Z e CX, i successi sarebbero 0. H è presente anche nelle baseline random e nel training aggiornato.

L'interfaccia dell'ambiente

- `action_space`: definisce le scelte disponibili. Ogni azione è un indice che identifica un gate e i qubit su cui applicarlo.
- `reset()`: inizia un nuovo episodio, svuota il circuito e gli angoli e restituisce l'osservazione iniziale di |00…0⟩.
- `step()`: aggiunge il gate scelto, ottimizza gli angoli e simula il circuito. Restituisce osservazione, reward, condizioni di fine episodio e metriche.
- `_get_obs()`: raccoglie ampiezze, progresso e storia del circuito, insieme alla maschera delle azioni ammesse, per la prossima decisione della policy.

> L'agente sceglie la **struttura discreta**. Gli angoli delle rotazioni sono determinati da un'ottimizzazione interna separata.

Implementazione: quantum_rl/2.environment.py.

Reward

La reward dipende da quanto diminuisce l'infedeltà rispetto al target, dal numero di gate utilizzati, dalle soglie di precisione raggiunte e dal successo finale del circuito.

Come si calcola il punteggio?

- **Avvicinarsi al target:** dà punti; allontanarsi li toglie. Se l'errore si dimezza, si ottengono circa **+6,93 punti**.
- **Aggiungere un gate:** costa sempre **−1 punto**.
- **Superare una soglia di precisione:** significa ottenere un risultato abbastanza preciso. Qui la soglia è **F > 0,99**: dà **+10 punti** solo la prima volta in ogni episodio, cioè in ogni tentativo di costruire il circuito.
- **Raggiungere il target:** dà **+100 punti** e **+5 per ogni gate disponibile ma non usato**. Esempio: con un limite di 30 gate e una soluzione di 3, si risparmiano 27 gate e si ricevono **+135 punti**.

Tutti i premi si aggiungono. C'è anche un premio per la brevità: al primo successo è 0; se usi tanti gate quanti la soluzione migliore precedente, è +50. Se ne usi meno, il premio sale; se ne usi di più, scende. Questo premio si riceve solo quando si raggiunge il target.

Estratto da step() · quantum_rl/2.environment.py · commenti esplicativi aggiunti

```
# Evita il logaritmo di zero.
epsilon = 1e-12
# Premia la riduzione dell'errore e sottrae il costo del gate.
reward = (
    -math.log(distance + epsilon)
    + math.log(self.prev_dist + epsilon)
) * self.loss_mult - self.step_penalty
# Memorizza l'errore per confrontarlo al prossimo step.
self.prev_dist = distance

# Successo: l'errore è sotto la soglia.
terminated = distance < self.threshold
# Stop: è stato raggiunto il numero massimo di gate.
truncated = len(self.current_gates) >= self.max_gates
```

> Il punteggio guida la ricerca di soluzioni brevi; non certifica che il circuito trovato sia il minimo globale.

Scelto il gate, quanto deve ruotare il qubit?

L’agente sceglie, per esempio, RY sul qubit 0. Per completare la mossa serve anche un angolo: lo cerca Adam, un ottimizzatore di PyTorch.

Che cosa succede a ogni mossa?

1. **L’agente aggiunge un gate.** La sequenza dei gate viene fissata per questa ottimizzazione.
2. **Il simulatore prova gli angoli.** PennyLane calcola lo stato ottenuto e quanto è vicino al target.
3. **Adam corregge gli angoli.** Usa il gradiente, che indica come modificarli per ridurre l’errore.
4. **Si conserva il risultato migliore.** L’ambiente calcola la reward e l’agente sceglie il prossimo gate.

Si regolano tutti gli angoli già presenti, anche dopo un CNOT. Il ciclo usa fino a 60 passi per tentativo di ottimizzazione.

Il ciclo interno, con commenti · _optimise_angles() · controlli omessi

```
# Adam modifica gli angoli del circuito.
optimiser = torch.optim.Adam([parameters], lr=self.angle_lr)

# Questi passaggi si ripetono nel ciclo:
# 1. Azzera i gradienti del passo precedente.
optimiser.zero_grad()
# 2. Simula il circuito con gli angoli attuali.
state = self._state_torch(parameters)
# 3. Misura la somiglianza con il target.
fidelity = torch.abs(
    torch.sum(torch.conj(target) * state)
) ** 2
# Errore da ridurre: 0 significa risultato perfetto.
loss = 1.0 - fidelity.real
# 4. Calcola come cambiare gli angoli e li aggiorna.
loss.backward()
optimiser.step()
```

Due compiti separati: l’agente sceglie quali gate usare; Adam regola quanto ruotano i qubit. Più passi per cercare gli angoli possono migliorare il risultato, ma richiedono più tempo.

03 · ALGORITMO DELL'AGENTE

Come impara l'agente a scegliere i gate?

PPO (Proximal Policy Optimization) usa i risultati dei tentativi per migliorare le scelte dell'agente.

L'agente prova
Sceglie gate e costruisce circuiti

→

Riceve la reward
Scopre quanto sono utili le scelte

→

PPO aggiorna la rete
Modifica i pesi per migliorare le scelte future

Il compito di PPO

Analizza le esperienze raccolte e rende più probabili le scelte che portano a risultati migliori.

Limita i cambiamenti troppo grandi, per rendere l'addestramento più stabile.

Il collegamento con il Transformer

Nel progetto, il **Transformer è la rete che osserva il circuito e propone il prossimo gate**.

PPO aggiorna i pesi di questa rete, così le sue proposte migliorano con l'esperienza.

> Nella prossima slide vediamo come il Transformer usa le informazioni del circuito per proporre un'azione.

PPO è implementato con Ray RLlib in 2.train.py.

Come il Transformer propone il prossimo gate

Riceve i numeri che descrivono il circuito, li mette in relazione e assegna una probabilità a ogni azione possibile: un gate applicato a determinati qubit.

Legge i dati
Ogni numero diventa un token con una posizione

→

Collega le informazioni
Due blocchi di elaborazione con attention

→

Riassume il circuito
Combina i token in una rappresentazione unica

→

Propone un'azione
Punteggi trasformati in probabilità

1 · Dai numeri ai token

Uno **scalare** è un singolo numero: per esempio una parte di un'ampiezza o l'errore attuale. Ogni numero viene trasformato in un **token**, un vettore di 128 valori che la rete può elaborare.

La **posizione** distingue il ruolo dei dati: un'ampiezza, il progresso o un elemento della storia dei gate.

2 · Mettere in relazione i dati

La **self-attention** permette a ciascun token di usare le informazioni degli altri. Due blocchi, ciascuno con 8 teste di attention, imparano relazioni tra stato, progresso e gate già applicati.

3 · Riassumere e decidere

La rete fa la media dei token elaborati e ottiene una descrizione complessiva del circuito. Da questo riassunto produce due risultati:

- **Policy:** un punteggio per ogni azione. Si escludono le azioni non ammesse e i punteggi diventano probabilità usate per scegliere il prossimo gate.
- **Critic:** una stima della reward futura, che aiuta PPO durante l'addestramento.

Nel training si campiona un'azione dalle probabilità; nella valutazione senza esplorazione si sceglie quella più probabile.

Architettura in quantum_rl/3.model.py. PPO aggiorna i pesi con cui il Transformer elabora i dati e assegna i punteggi alle azioni.

04 · I LIVELLI DELL'ESPERIMENTO

Ciclo di addestramento

Step → episodio → iterazione → run

Livello	Significato	Che cosa succede nel progetto?
Step	Una singola mossa	L'agente sceglie un gate. L'ambiente lo aggiunge, regola gli angoli delle rotazioni e calcola lo stato ottenuto e la reward.
Episodio	Un tentativo di costruire il circuito	Si parte da $\|00\ldots0\rangle$ senza gate e si eseguono più step. Il tentativo finisce quando il target è raggiunto o si esaurisce il numero di gate disponibili.
Iterazione	Un ciclo in cui l'agente impara	I worker raccolgono mosse e reward da più tentativi. PPO usa questi dati per aggiornare i pesi del Transformer e migliorare le scelte successive.
Run	Un intero addestramento	Si crea una nuova rete con pesi iniziali casuali e si eseguono tutte le iterazioni previste. Alla fine si valuta ciò che l'agente ha imparato.

Configurazione scelta per il training

10 run · 10 iterazioni per run · 2 worker · massimo 30 gate per episodio · 10 episodi di valutazione.

Perché ripetere le run?

Per osservare la variabilità: una singola soluzione riuscita non dimostra che l'addestramento sia affidabile.

04 · ESECUZIONE SUL CLUSTER UNIPR

Dal computer locale al job HPC

PC locale → SSH / login → SLURM → Nodo di calcolo

Organizzazione del lavoro

- Accesso al cluster e alla cartella Definitivo.
- Attivazione dell'ambiente Conda `quantum-rl`.
- Invio del job con `sbatch job_train.slurm`.
- Controllo della coda con `squeue` e dei log di output ed errore.

Uso delle risorse

- Il job continua anche quando il PC locale è spento.
- Ray parallelizza la raccolta delle esperienze tramite i worker.
- Richieste di risorse e disponibilità influenzano l'attesa in coda.
- Il codice limita i thread PyTorch per evitare contesa tra worker.

Il job_train.slurm presente al momento della preparazione usa target random a 3 qubit: 10 run, 10 iterazioni, 30 gate, 2 worker, 10 episodi di valutazione; richiesta di 16 GB e limite di un giorno. Le run nello script ripetuto sono sequenziali.

05 · PRIMA DEL TRAINING · BASELINE CASUALE

Stato target: Bell

baseline_bell_summary.csv · Esempio completato con righe duplicate (*) · Gate casuali: H, X, Y, Z, CX

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	✅ Successo	1.00	13	8.70	`CX(1, 0) -> X(1) -> H(1) -> ... (13 gate)`
2	✅ Successo	1.00	6	9.40	`Y(0) -> Y(0) -> H(0) -> Z(0) -> CX(0, 1) -> Z(1)`
3	✅ Successo	1.00	10	9.00	`X(1) -> H(0) -> Y(1) -> ... (10 gate)`
4	❌ Fallito (0)	0.25	30	-0.50	`Z(0) -> Z(0) -> Y(1) -> ... (30 gate)`
5	✅ Successo	1.00	12	8.80	`Y(1) -> Z(0) -> X(0) -> ... (12 gate)`
6	✅ Successo	1.00	24	7.60	`Y(1) -> Y(1) -> Y(0) -> ... (24 gate)`
7	❌ Fallito (0)	0.25	30	-0.50	`CX(1, 0) -> X(0) -> X(0) -> ... (30 gate)`
8	✅ Successo	1.00	6	9.40	`Y(0) -> Y(0) -> H(0) -> Z(0) -> CX(0, 1) -> Z(1)`
9	✅ Successo	1.00	24	7.60	`Y(1) -> Y(1) -> Y(0) -> ... (24 gate)`
10	❌ Fallito (0)	0.25	30	-0.50	`Z(0) -> Z(0) -> Y(1) -> ... (30 gate)`
Media	**70.0%**	**0.7750**	**18.1**	**5.94**	**–**

- **Circuito ideale da raggiungere:** H(0) → CX(0, 1).
- **Successi:** 7/10.

05 · PRIMA DEL TRAINING · BASELINE CASUALE

Stato target: GHZ

baseline_ghz_summary.csv · Riga 8 duplicata dalla 10 · Gate casuali: H, X, Y, Z, CX

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	✅ Successo	1.00	13	8.70	`CX(1, 0) -> X(1) -> H(1) -> ... (13 gate)`
2	✅ Successo	1.00	6	9.40	`Y(0) -> Y(0) -> H(0) -> Z(0) -> ... (6 gate)`
3	✅ Successo	1.00	10	9.00	`X(1) -> H(0) -> Y(1) -> ... (10 gate)`
4	❌ Fallito (0)	0.25	30	-0.50	`Z(0) -> Z(0) -> Y(1) -> ... (30 gate)`
5	✅ Successo	1.00	12	8.80	`Y(1) -> Z(0) -> X(0) -> ... (12 gate)`
6	✅ Successo	1.00	24	7.60	`Y(1) -> Y(1) -> Y(0) -> ... (24 gate)`
7	❌ Fallito (0)	0.25	30	-0.50	`CX(1, 0) -> X(0) -> X(0) -> ... (30 gate)`
8	❌ Fallito (0)	0.125	30	-1.75	`H(0) -> Y(2) -> CX(0, 2) -> ... (30 gate)`
9	❌ Fallito (0)	0.25	30	-0.50	`Z(1) -> X(0) -> Y(1) -> ... (30 gate)`
10	❌ Fallito (0)	0.125	30	-1.75	`H(0) -> Y(2) -> CX(0, 2) -> ... (30 gate)`
Media	**50.0%**	**0.6000**	**24.5**	**3.85**	**–**

- **Circuito ideale da raggiungere:** H(0) → CX(0, 1) → CX(0, 2).
- **Successi:** 5/10.

05 · PRIMA DEL TRAINING · BASELINE CASUALE

Stato target: Random a 2 qubit

baseline_random_target_2_qubit_normalized_summary.csv · Righe 8–10: copie illustrative (*) · Gate casuali: H, X, Y, Z, CX

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	❌ Fallito (0)	0.4394	30	1.3939	`Y(0) -> Y(1) -> CX(0, 1) -> ... (30 gate)`
2	❌ Fallito (0)	0.0076	30	-2.9242	`CX(1, 0) -> CX(1, 0) -> Z(0) -> ... (30 gate)`
3	❌ Fallito (0)	0.2424	30	-0.5758	`H(0) -> H(1) -> Z(0) -> ... (30 gate)`
4	❌ Fallito (0)	0.3030	30	0.0303	`X(0) -> Z(0) -> Z(1) -> ... (30 gate)`
5	❌ Fallito (0)	0.3030	30	0.0303	`X(0) -> CX(0, 1) -> Y(0) -> ... (30 gate)`
6	❌ Fallito (0)	0.1515	30	-1.4848	`Y(0) -> H(0) -> Z(0) -> ... (30 gate)`
7	❌ Fallito (0)	0.0303	30	-2.6970	`Z(0) -> Z(1) -> Y(0) -> ... (30 gate)`
8	❌ Fallito (0)	0.2424	30	-0.5758	`H(0) -> H(1) -> Z(0) -> ... (30 gate)`
9	❌ Fallito (0)	0.1515	30	-1.4848	`Y(0) -> H(0) -> Z(0) -> ... (30 gate)`
10	❌ Fallito (0)	0.3030	30	0.0303	`X(0) -> CX(0, 1) -> Y(0) -> ... (30 gate)`
Media	**0.0%**	**0.2174**	**30.0**	**-0.8258**	**–**

- **Circuito ideale da raggiungere:**
  `RY(1.66183111, 0) → RY(1.45654838, 1) → CNOT(0, 1) → RY(-0.22558896, 1) → CNOT(0, 1) → RZ(1.73167160, 0) → RZ(-0.16087528, 1) → CNOT(0, 1) → RZ(-1.26802399, 1) → CNOT(0, 1)`
  Sequenza di gate ideale rispetto al target random a 2 qubit scelto
- **Successi:** 0/10.
- **Motivo del completo insuccesso:** I gate H, X, Y, Z e CX non permettono di preparare qualsiasi stato puro arbitrario: servono anche rotazioni con angoli continui.

Stato target: Random a 3 qubit

baseline_random_target_3_qubit_normalized_summary.csv · Righe 8–10: copie illustrative (*) · Gate casuali: H, X, Y, Z, CX

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	❌ Fallito (0)	0.1335	30	-1.6651	`Y(1) -> X(2) -> CX(1, 0) -> ... (30 gate)`
2	❌ Fallito (0)	0.0313	30	-2.6867	`Z(1) -> CX(0, 2) -> Y(2) -> ... (30 gate)`
3	❌ Fallito (0)	0.1287	30	-1.7132	`Y(1) -> CX(1, 0) -> X(1) -> ... (30 gate)`
4	❌ Fallito (0)	0.0590	30	-2.4095	`H(2) -> X(1) -> Y(0) -> ... (30 gate)`
5	❌ Fallito (0)	0.0618	30	-2.3819	`Z(0) -> CX(1, 2) -> Y(1) -> ... (30 gate)`
6	❌ Fallito (0)	0.0416	30	-2.5844	`Z(2) -> Z(0) -> CX(2, 1) -> ... (30 gate)`
7	❌ Fallito (0)	0.1483	30	-1.5168	`Y(0) -> CX(0, 2) -> Z(2) -> ... (30 gate)`
8	❌ Fallito (0)	0.1483	30	-1.5168	`Y(0) -> CX(0, 2) -> Z(2) -> ... (30 gate)`
9	❌ Fallito (0)	0.1287	30	-1.7132	`Y(1) -> CX(1, 0) -> X(1) -> ... (30 gate)`
10	❌ Fallito (0)	0.0416	30	-2.5844	`Z(2) -> Z(0) -> CX(2, 1) -> ... (30 gate)`
Media	**0.0%**	**0.0923**	**30.0**	**-2.0772**	**–**

- **Circuito ideale da raggiungere:**

  `RY(1.49030847, 0) → RY(1.36854330, 1) → CNOT(0, 1) → RY(0.27256278, 1) → CNOT(0, 1) → RY(1.41649246, 2) → CNOT(1, 2) → RY(0.14036059, 2) → CNOT(0, 2) → RY(-0.10326143, 2) → CNOT(1, 2) → RY(-0.02543737, 2) → CNOT(0, 2) → RZ(-0.77472220, 0) → RZ(-1.86746063, 1) → CNOT(0, 1) → RZ(0.10445689, 1) → CNOT(0, 1) → RZ(1.15683871, 2) → CNOT(1, 2) → RZ(-1.36671372, 2) → CNOT(0, 2) → RZ(1.30815857, 2) → CNOT(1, 2) → RZ(0.85989662, 2) → CNOT(0, 2)`

  Sequenza di gate ideale rispetto al target random a 3 qubit scelto

- **Successi:** 0/10.
- **Motivo del completo insuccesso:** I gate H, X, Y, Z e CX non permettono di preparare qualsiasi stato puro arbitrario: servono anche rotazioni con angoli continui.

05 · RISULTATI DELLA BASELINE CASUALE

Che cosa emerge?

- Bell può essere trovato per caso, ma con gate ridondanti.
- Nella tabella preliminare GHZ riesce in 5 prove su 10, ma con circuiti ridondanti.
- I gate della baseline generano stati stabilizer: non coprono gli stati puri arbitrari.

> Da questo momento, fino al training aggiornato, non useremo il gate H: l'agente avrà a disposizione solo RX, RY, RZ e CNOT. Hadamard verrà introdotto nella versione aggiornata per aiutare a superare il blocco osservato su GHZ.

Bell: successo nell'80% delle run

8 / 10
Run riuscite

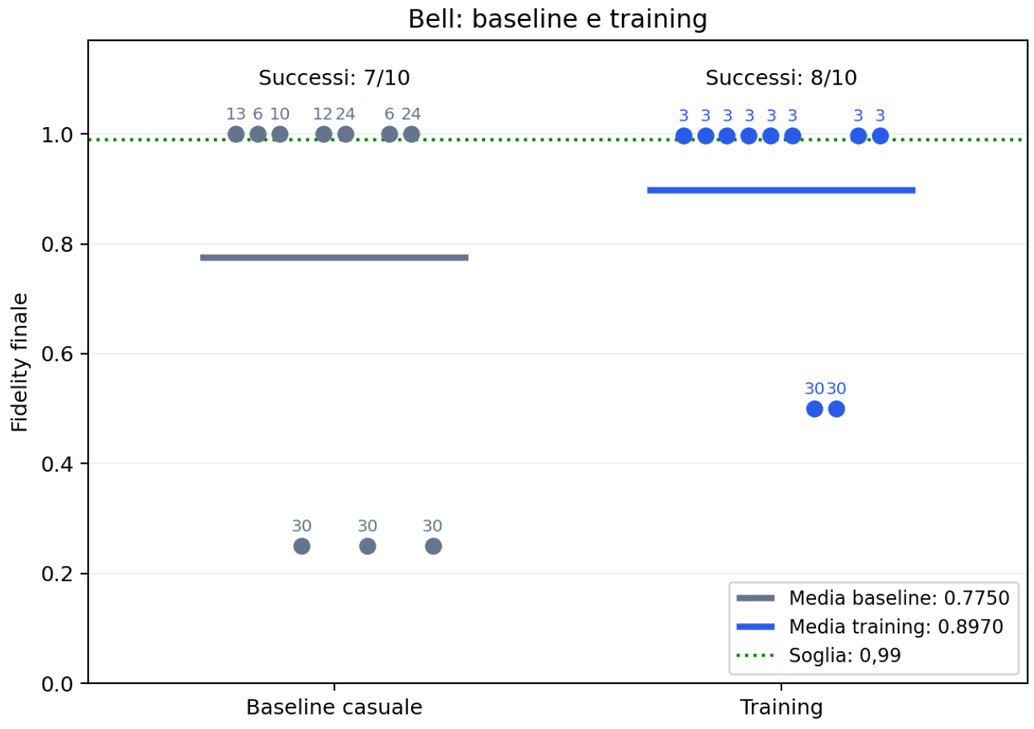
0,8970
Fidelity media

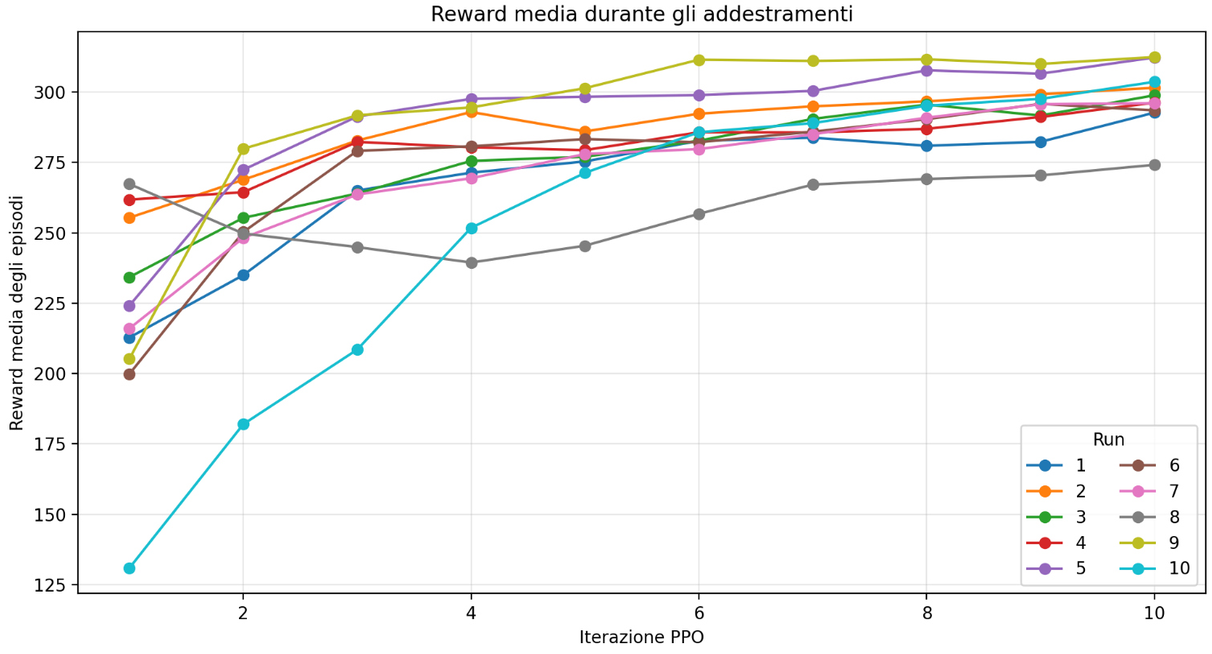
≈ 1 h 32 min
Tempo totale di training · 10 run

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	✅ Successo	0.9963	3	291.00	`CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)`
2	✅ Successo	0.9963	3	291.00	`CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)`
3	✅ Successo	0.9963	3	291.00	`CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)`
4	✅ Successo	0.9963	3	291.00	`CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)`
5	✅ Successo	0.9963	3	291.00	`CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)`
6	✅ Successo	0.9963	3	291.00	`CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)`
7	❌ Fallito (0)	0.5000	30	-30.00	`RY(0.000000, 1) -> RY(0.000000, 1) -> ... (30 gate)`
8	❌ Fallito (0)	0.5000	30	-30.00	`RY(0.000000, 0) -> RY(0.000000, 0) -> ... (30 gate)`
9	✅ Successo	0.9963	3	291.00	`CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)`
10	✅ Successo	0.9963	3	291.00	`CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)`
Media	**80.0%**	**0.8970**	**—**	**226.80**	**–**

Circuito trovato: CNOT(1, 0) → RY(1,692911, 1) → CNOT(1, 0)

Bell: confronto e andamento del training





Run senza successo finale (migliore fidelity < 0,99): 7, 8

GHZ: il primo training si blocca

0 / 10
Run riuscite

0,5000
Fidelity media

≈ 1 h 46 min
Tempo totale di training · 10 run

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(0, 2) -> RY(0.000000, 0) -> ... (30 gate)`
2	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(2, 1) -> RY(0.000000, 2) -> ... (30 gate)`
3	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(1, 2) -> CNOT(0, 1) -> ... (30 gate)`
4	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(1, 0) -> CNOT(1, 2) -> ... (30 gate)`
5	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(1, 2) -> CNOT(0, 2) -> ... (30 gate)`
6	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(2, 1) -> CNOT(0, 2) -> ... (30 gate)`
7	❌ Fallito (0)	0.5000	30	-30.00	`RY(0.000000, 2) -> RY(0.000000, 2) -> ... (30 gate)`
8	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(2, 0) -> RZ(0.000000, 1) -> ... (30 gate)`
9	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(0, 2) -> RY(0.000000, 0) -> ... (30 gate)`
10	❌ Fallito (0)	0.5000	30	-30.00	`CNOT(1, 2) -> CNOT(1, 0) -> ... (30 gate)`
Media	**0.0%**	**0.5000**	**—**	**-30.00**	**–**

05 · IL PROBLEMA

Perché Bell riesce all’80% e GHZ resta allo 0%?

Entrambi partono da fidelity 0,5. GHZ richiede però di collegare tre qubit: servono più gate prima che la rotazione iniziale diventi utile.

Come nasce il blocco?

1. L’episodio parte da **|000⟩**, senza gate.
2. L’agente aggiunge RY: il suo **angolo iniziale è 0**, quindi non ruota il qubit.
3. Adam cerca come migliorarlo, ma a zero il **gradiente è nullo**: non indica una direzione utile.
4. Finché mancano i CNOT necessari, una piccola rotazione peggiora la fidelity. Adam mantiene o riporta l’angolo a zero.

Perché GHZ è più difficile?

Per Bell basta una rotazione seguita da **un CNOT**. Per GHZ occorrono **due CNOT** che colleghino tutti e tre i qubit.

L’agente deve quindi costruire una sequenza più lunga prima di ottenere il miglioramento. Può invece ripetere rotazioni nulle e CNOT inutili.

Risultato osservato su GHZ: l’angolo resta a zero, la fidelity rimane a 0,5 e il circuito arriva a 30 gate senza successo.

05 · LE SOLUZIONI

Come superare il blocco degli angoli a zero?

Non affidarsi a un solo punto di partenza: provare angoli diversi e aiutare l'agente a costruire la sovrapposizione necessaria.

1 · Più partenze: multi-start

1. Provare più angoli iniziali casuali: negli appunti, **5 tentativi tra 0 e 2π**.
2. Per ogni tentativo, usare Adam per regolare gli angoli del circuito.
3. Confrontare la fidelity ottenuta e **conservare il risultato migliore**.

Partire lontano da zero può evitare un punto con gradiente nullo quando la struttura consente di migliorare.

Il multi-start da solo non basta se mancano i gate di entanglement: il circuito deve avere una struttura utile.

2 · Hadamard e meno mosse inutili

- **Aggiungere H:** crea una sovrapposizione fissa, senza un angolo che Adam possa riportare a zero.
- **Riottimizzare dopo i CNOT:** una volta collegati i qubit, gli angoli possono diventare utili per raggiungere il target.
- **Bloccare azioni ridondanti:** le maschere aiutano a evitare ripetizioni e CNOT senza effetto.

05 · TRAINING AGGIORNATO

GHZ: da 0 a 8 run riuscite su 10

8 / 10
Run riuscite

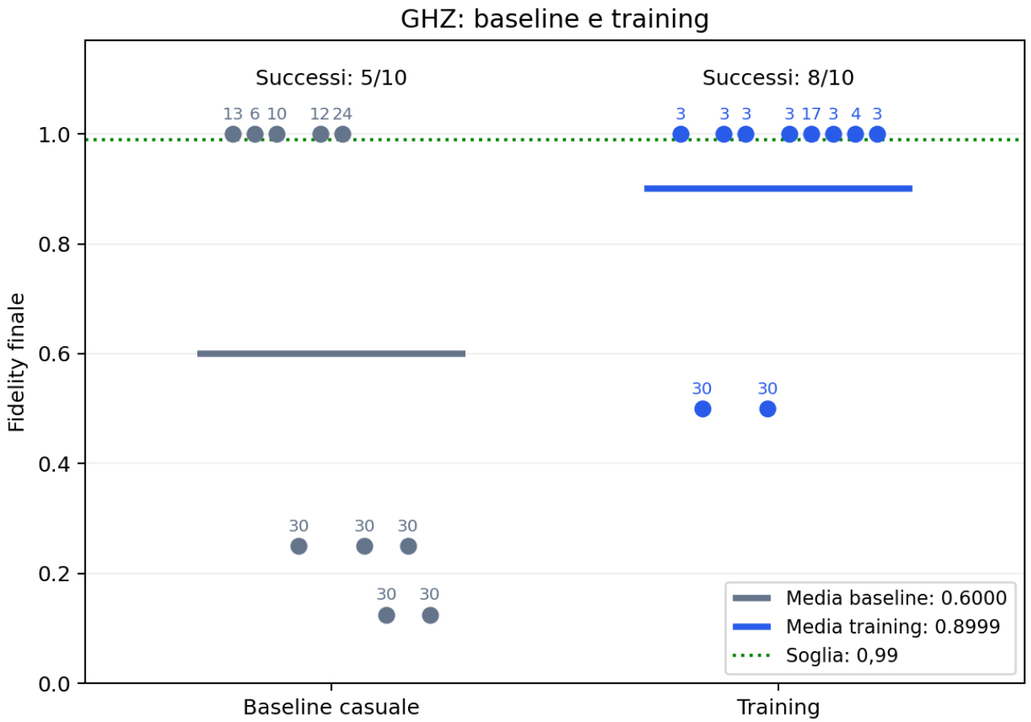
0,8999
Fidelity media

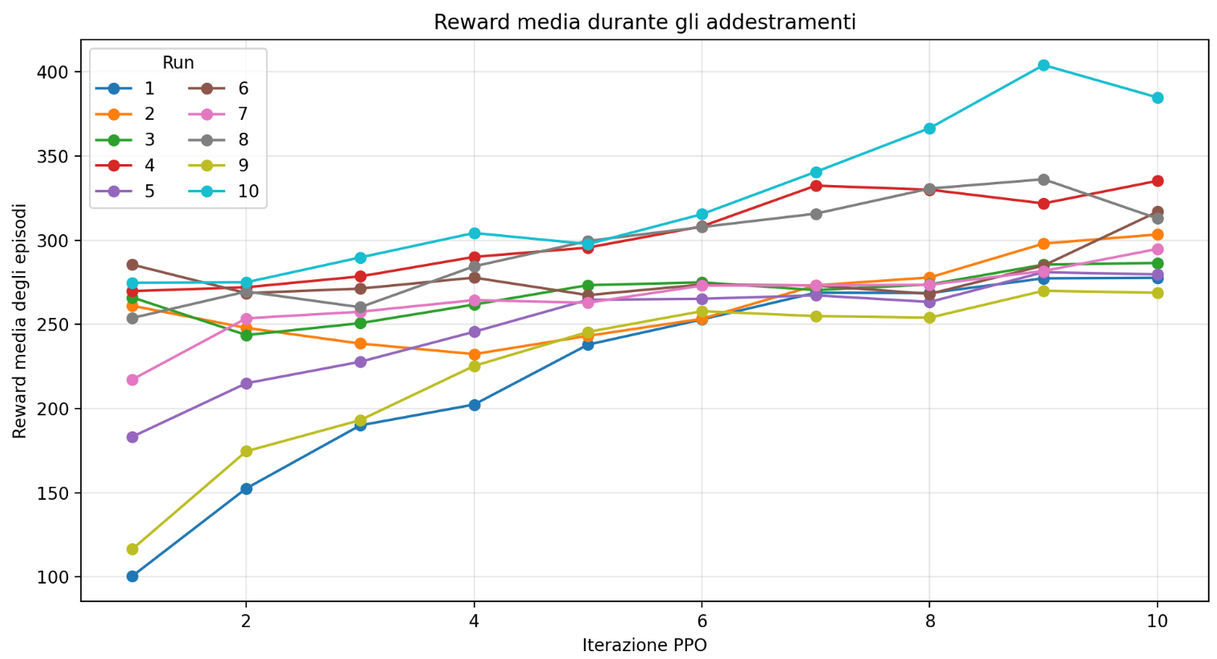
≈ 4 h 16 min
Tempo totale di training · 10 run

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	✅ Successo	0.9995	3	311.73	`RY(1.527506, 1) -> CNOT(1, 2) -> CNOT(2, 0)`
2	❌ Fallito (0)	0.5000	30	-30.00	`RY(0.000000, 0) -> CNOT(0, 2) -> ... (30 gate)`
3	✅ Successo	1.0000	3	511.38	`H(1) -> CNOT(1, 2) -> CNOT(2, 0)`
4	✅ Successo	1.0000	3	511.38	`H(2) -> CNOT(2, 0) -> CNOT(0, 1)`
5	❌ Fallito (0)	0.5000	30	-30.00	`RX(0.000000, 1) -> CNOT(1, 0) -> ... (30 gate)`
6	✅ Successo	0.9995	3	311.73	`RY(1.527506, 2) -> CNOT(2, 1) -> CNOT(1, 0)`
7	✅ Successo	0.9995	17	227.73	`RY(1.527506, 1) -> RZ(0.000000, 1) -> CNOT(1, 0)...`
8	✅ Successo	1.0000	3	511.38	`H(1) -> CNOT(1, 2) -> CNOT(2, 0)`
9	✅ Successo	1.0000	4	505.38	`RX(0.000000, 2) -> H(0) -> CNOT(0, 1) -> CNOT(0, 2)`
10	✅ Successo	1.0000	3	511.38	`H(1) -> CNOT(1, 2) -> CNOT(2, 0)`
Media	**80.0%**	**0.8999**	**—**	**334.21**	**–**

Circuito trovato: H(1) → CNOT(1, 2) → CNOT(2, 0)

GHZ: confronto e andamento del training





Run senza successo finale (migliore fidelity < 0,99): 2, 5

05 · TRAINING AGGIORNATO

Random a 2 qubit: il 50% delle run riesce

5 / 10
Run riuscite

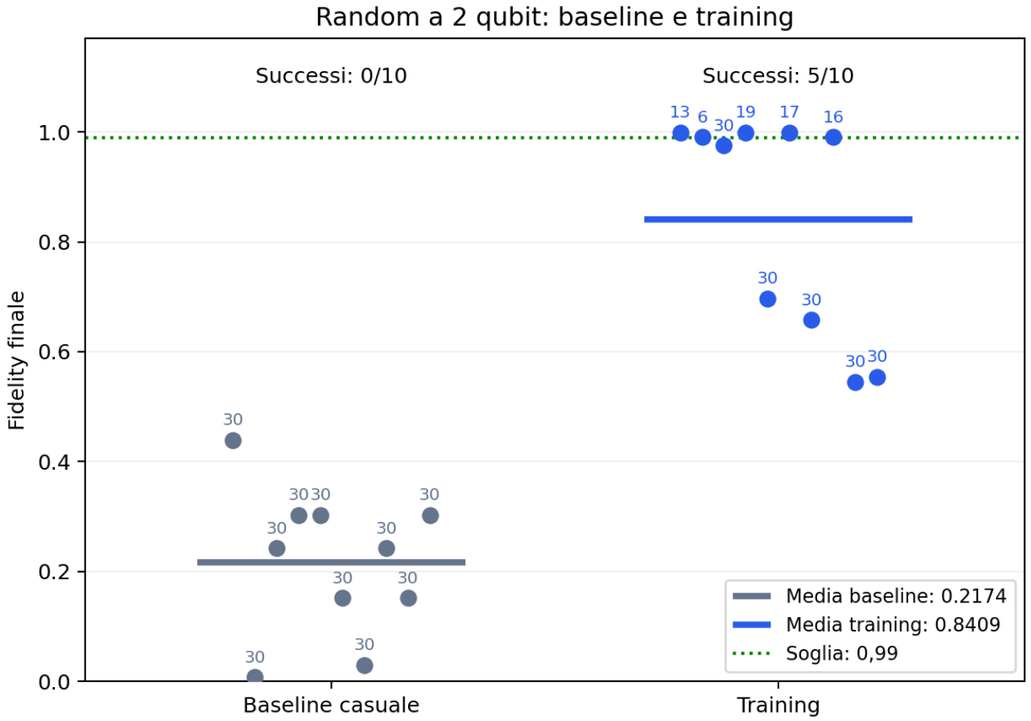
0,8409
Fidelity media

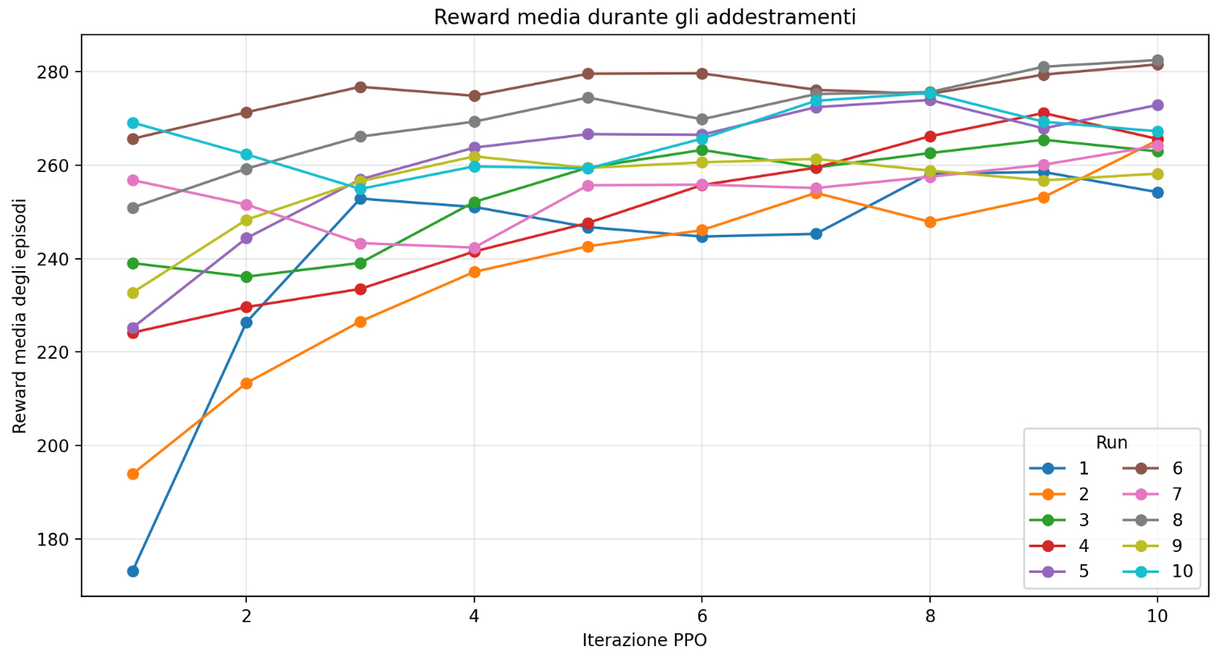
≈ 5 h 30 min
Tempo totale di training · 10 run

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	✅ Successo	0.9991	13	3.78	`RY(-2.753359, 1) -> CNOT(1, 0) -> RY(0.258590, 1) -> ...`
2	✅ Successo	0.9907	6	264.72	`RY(1.349271, 0) -> CNOT(0, 1) -> RZ(0.327477, 1) -> RY(1.465698, 0) -> CNOT(0, 1) -> RZ(1.447523, 1)`
3	❌ Fallito (0)	0.9755	30	3.49	`RY(1.474781, 0) -> CNOT(0, 1) -> RZ(1.383262, 0) -> ... (30 gate)`
4	✅ Successo	0.9990	19	208.22	`RY(1.297238, 0) -> RY(2.208652, 1) -> CNOT(0, 1) -> ...`
5	❌ Fallito (0)	0.6970	30	-21.67	`RX(-0.858371, 0) -> CNOT(0, 1) -> RX(2.685569, 0) -> ... (30 gate)`
6	✅ Successo	0.9993	17	228.72	`RY(-1.221655, 1) -> CNOT(1, 0) -> RY(1.143484, 1) -> ...`
7	❌ Fallito (0)	0.6582	30	-22.88	`RX(-1.862911, 0) -> RX(3.015778, 1) -> H(1) -> ... (30 gate)`
8	✅ Successo	0.9902	16	39.31	`RY(-1.091360, 1) -> CNOT(1, 0) -> RY(0.218657, 1) -> ...`
9	❌ Fallito (0)	0.5455	30	-25.73	`RY(0.314349, 0) -> RZ(-2.278639, 0) -> ... (30 gate)`
10	❌ Fallito (0)	0.5546	30	-25.52	`RY(2.657912, 0) -> RZ(-1.323871, 0) -> ... (30 gate)`
Media	**50.0%**	**0.8409**	**—**	**65.25**	**–**

Circuito più breve trovato: RY(1,349271, 0) → CNOT(0, 1) → RZ(0,327477, 1) → RY(1,465698, 0) → CNOT(0, 1) → RZ(1,447523, 1)

Random a 2 qubit: confronto e andamento del training





Run senza successo finale (migliore fidelity < 0,99): 3, 5, 7, 9, 10

05 · TRAINING AGGIORNATO

Random a 3 qubit: miglioramento senza successo

0 / 10
Run riuscite

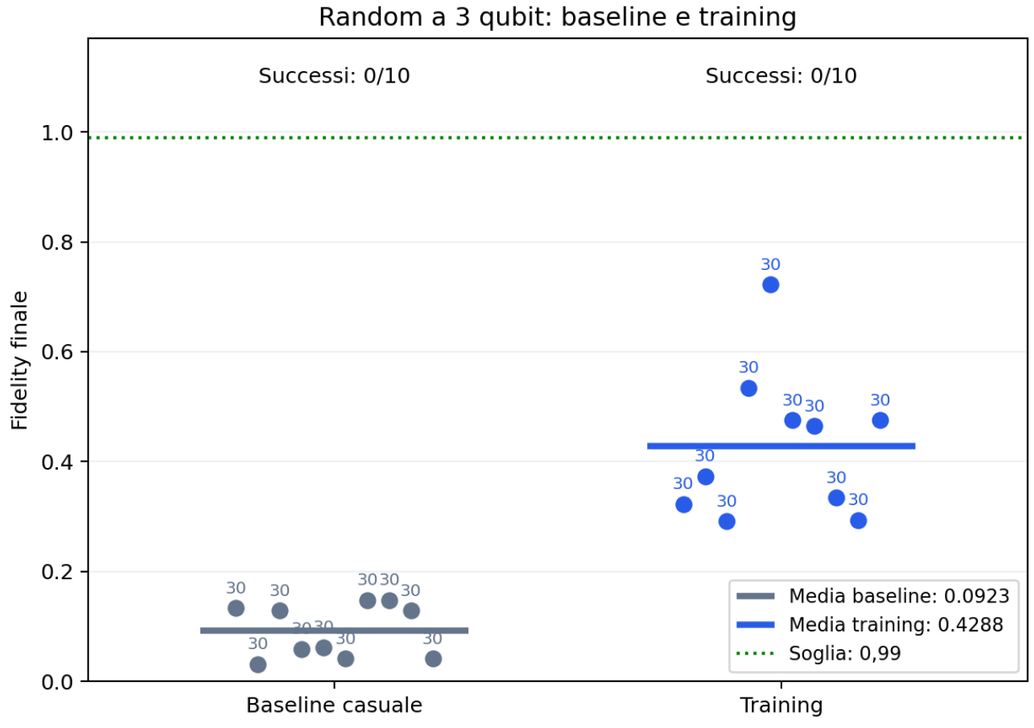
0,4288
Fidelity media

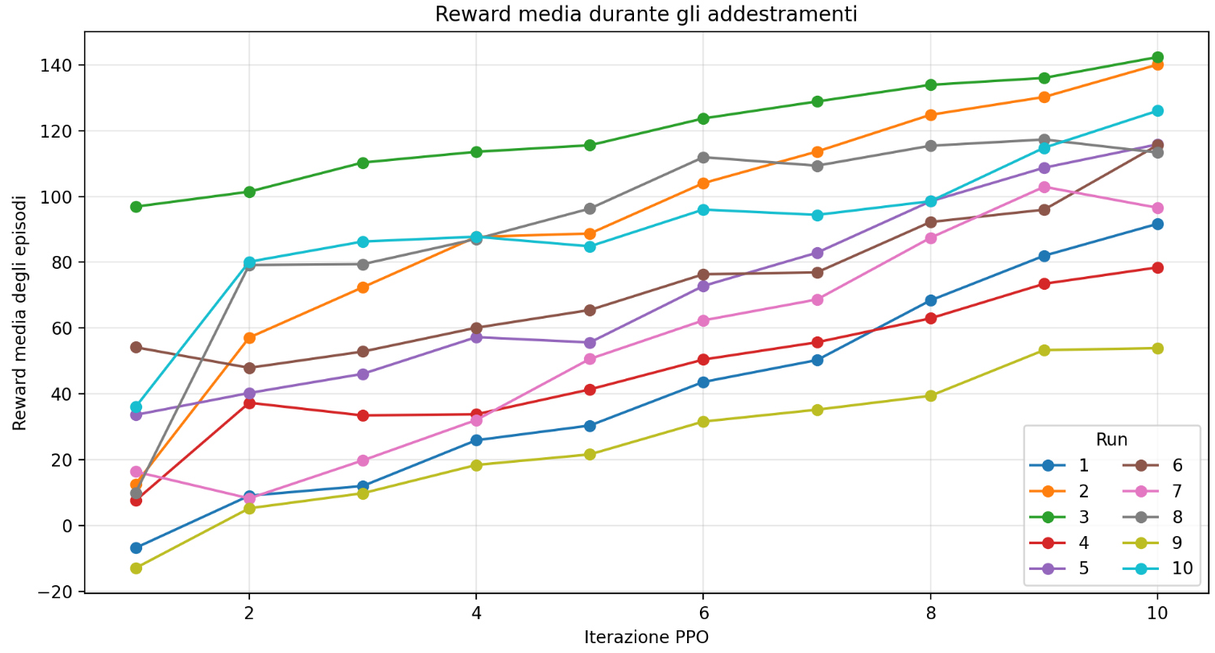
≈ 12 h 39 min
Tempo totale di training · 10 run

Run	Esito	Fidelity finale	Gate	Reward totale	Circuito trovato
1	❌ Fallito (0)	0.3232	30	-27.64	`RY(-1.452811, 2) -> CNOT(2, 0) -> RY(2.666358, 2)... (30 gate)`
2	❌ Fallito (0)	0.3737	30	-26.87	`RY(-0.052656, 1) -> RX(2.052943, 0) -> RX(-1.882298, 2)... (30 gate)`
3	❌ Fallito (0)	0.2917	30	-28.10	`RY(3.068356, 0) -> RX(1.578991, 0) -> RY(0.325931, 0)... (30 gate)`
4	❌ Fallito (0)	0.5341	30	-24.44	`H(2) -> RY(2.805568, 2) -> H(2) -> RY(3.018255, 2)... (30 gate)`
5	❌ Fallito (0)	0.7221	30	-21.22	`RY(-1.003208, 2) -> RX(-1.803071, 1) -> RY(-1.313247, 0)... (30 gate)`
6	❌ Fallito (0)	0.4752	30	-25.10	`RX(0.237595, 1) -> RY(-1.732116, 2) -> RY(-1.739242, 0)... (30 gate)`
7	❌ Fallito (0)	0.4650	30	-25.29	`RY(-1.343494, 2) -> RX(-1.818069, 0) -> RZ(-1.546965, 0)... (30 gate)`
8	❌ Fallito (0)	0.3341	30	-27.48	`RY(-3.118705, 0) -> CNOT(0, 1) -> RY(-3.127674, 0)... (30 gate)`
9	❌ Fallito (0)	0.2940	30	-28.07	`H(0) -> RY(-1.501816, 1) -> CNOT(1, 2) -> CNOT(2, 1)... (30 gate)`
10	❌ Fallito (0)	0.4751	30	-27.17	`RY(2.366329, 0) -> RY(2.911689, 1) -> CNOT(0, 1)... (30 gate)`
Media	**0.0%**	**0.4288**	**—**	**-26.14**	**–**

Migliore tentativo · F = 0,7221: RY(−1,003208, 2) → RX(−1,803071, 1) → RY(−1,313247, 0) → …

Random a 3 qubit: confronto e andamento del training





Run senza successo finale (migliore fidelity < 0,99): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

06 · POSSIBILI SVILUPPI

Come ottenere un agente più intelligente?

L'obiettivo è fargli trovare il target più spesso e con meno gate, soprattutto nel caso random a 3 qubit.

1 · Aumentare le iterazioni

Passare, per esempio, da **10 a 100 iterazioni**. PPO avrebbe più occasioni per aggiornare il Transformer e correggere le scelte poco utili.

2 · Raccogliere più esperienze

Far costruire più circuiti durante il training, raccogliendo più mosse e reward prima degli aggiornamenti. L'agente potrebbe imparare da una maggiore varietà di tentativi.

Più episodi di valutazione servono solo a misurare meglio i risultati: non insegnano nuove strategie.

3 · Migliorare la ricerca degli angoli

Provare più angoli iniziali casuali o aumentare i passi di Adam. Questo può aiutare a sfruttare meglio una buona sequenza di gate.

4 · Favorire l'esplorazione

Regolare l'esplorazione di PPO per provare sequenze diverse ed evitare di ripetere sempre le stesse mosse. Valutare anche se un budget di gate maggiore aiuta nei target più difficili.

Come scegliere: modificare un parametro alla volta e confrontare più run. Più calcolo richiede più tempo e non garantisce un miglioramento: vanno controllati successi, fidelity e gate usati.

Grazie per l’attenzione